

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

”Сибирский государственный университет телекоммуникаций и
информатики”

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по лабораторной работе №8
по дисциплине
Системы искусственного интеллекта
Полиномиальная регрессия

Студент:
Группа ИА-331

И.А. Иванов

Преподаватель:
К.И. Брагин

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ №8	4
1.1 Выбор набора данных	4
1.2 Теоретическое обоснование	4
1.2.1 Математическая модель полиномиальной регрессии ...	4
1.2.2 Особенности работы с датасетом Abalone	5
1.3 Ход выполнения работы	5
1.3.1 Шаг 1: Загрузка и первичный анализ данных	5
1.3.2 Шаг 2: Визуализация исходных данных	6
1.3.3 Шаг 3: Линейная регрессия (полином степени 1).....	6
1.3.4 Шаг 4: Полиномиальная регрессия различных степеней	7
1.3.5 Шаг 5: Визуализация моделей различных степеней	8
1.3.6 Шаг 6: Оценка на тестовой выборке.....	9
1.3.7 Шаг 7: Анализ переобучения	10
1.3.8 Шаг 8: Выбор оптимальной степени полинома	10
1.3.9 Шаг 9: График MSE и R^2 от степени полинома	11
1.3.10 Шаг 10: Предсказание для нового объекта	12
1.3.11 Шаг 11: Применение пайплайна	13
1.4 Сравнение моделей	14
1.5 Анализ результатов	14
1.5.1 Анализ зависимости качества от степени полинома.....	14
1.5.2 Интерпретация результатов для Abalone	15
1.6 Контрольные вопросы	15
1.7 Листинг программы	16

ЦЕЛИ И ЗАДАЧИ

Цель лабораторной работы: научиться применять разработанный пайплайн для тиражирования кода с целью решения задачи полиномиальной регрессии.

Основные задачи:

- получение навыков рефакторинга кода в проектах машинного обучения;
- изучение поведения модели полиномиальной регрессии при изменении степени полинома;
- исследование свойств набора данных в рамках задачи полиномиальной регрессии.

Теоретическое обоснование:

Полиномиальная регрессия является частным случаем множественной линейной регрессии, в которой связь между независимой переменной x и зависимой переменной y моделируется полиномом степени n . Модель полиномиальной регрессии имеет вид:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n + \varepsilon \quad (1)$$

где β_i — коэффициенты модели, n — степень полинома, ε — ошибка модели.

Преимущество полиномиальной регрессии заключается в способности аппроксимировать нелинейные зависимости между переменными. Однако при выборе степени полинома необходимо соблюдать баланс: слишком низкая степень приводит к недообучению (underfitting), а слишком высокая — к переобучению (overfitting).

В качестве набора данных для выполнения лабораторной работы используется набор данных **Position Salaries**, который содержит информацию о должностях и соответствующих уровнях и зарплатах. Данный набор демонстрирует нелинейную зависимость между уровнем должности и зарплатой.

ВЫПОЛНЕНИЕ ЛАБОРАТОРНОЙ РАБОТЫ №8

1.1 Выбор набора данных

Для выполнения лабораторной работы используется набор данных **Abalone**, который содержит информацию о морских моллюсках. Этот набор данных выбран для демонстрации нелинейной зависимости между физическими характеристиками моллюсков и их возрастом.

- Количество записей: 4177
- Количество признаков: 8
- Целевая переменная: Rings (количество колец на раковине, определяющее возраст)
- Используемый признак: Length (длина моллюска в мм)

Данный набор данных демонстрирует нелинейную зависимость: длина моллюска увеличивается с возрастом, но темп роста замедляется.

1.2 Теоретическое обоснование

1.2.1 Математическая модель полиномиальной регрессии

Пусть задана выборка $\{(x_i, y_i)\}_{i=1}^m$, где x_i — значение признака (длина), y_i — целевая переменная (количество колец). Модель полиномиальной регрессии степени d имеет вид:

$$\hat{y}_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \dots + \beta_d x_i^d \quad (2)$$

Параметры модели $\beta = (\beta_0, \beta_1, \dots, \beta_d)$ находятся методом наименьших квадратов (МНК) путем минимизации функции потерь:

$$Q(\beta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 \rightarrow \min_{\beta} \quad (3)$$

1.2.2 Особенности работы с датасетом Abalone

Датасет Abalone имеет следующие статистические характеристики:

- Диапазон длины: от 0.075 до 0.815 мм
- Диапазон колец: от 1 до 29
- Распределение колец имеет положительную асимметрию

1.3 Ход выполнения работы

1.3.1 Шаг 1: Загрузка и первичный анализ данных

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 from sklearn.linear_model import LinearRegression
5 from sklearn.preprocessing import PolynomialFeatures
6 from sklearn.metrics import mean_squared_error, r2_score
7 from sklearn.model_selection import train_test_split
8
9 dataset = pd.read_csv('abalone.csv')
10 print("      5      :")
11 print(dataset.head())
12 print(f"\      n      : {dataset.shape}")
13
14 X = dataset.iloc[:, 1:2].values #      Length
15 y = dataset.iloc[:, -1].values #      Rings
```

Листинг 1.1 — Загрузка и анализ данных Abalone

Результат:

```
      5      :
   Sex  Length  Diameter  Height  Whole weight  Shucked weight  Viscer
0    M    0.455    0.365    0.095         0.514         0.2245
1    M    0.350    0.265    0.090         0.2255         0.0995
2    F    0.530    0.420    0.135         0.677         0.2565
3    M    0.440    0.365    0.125         0.516         0.2155
4    I    0.330    0.255    0.080         0.205         0.0895
```

: (4177, 9)

1.3.2 Шаг 2: Визуализация исходных данных

```
1 plt.figure(figsize=(10, 6))
2 plt.scatter(X, y, color='red', s=10, alpha=0.5, label='Исходные данные')
3 plt.xlabel('Длина моллюска (Length)', fontsize=12)
4 plt.ylabel('Количество колец (Rings)', fontsize=12)
5 plt.title('Зависимость возраста моллюска от длины', fontsize=14)
6 plt.grid(True, alpha=0.3)
7 plt.legend()
8 plt.savefig('lab8_initial_data.png', dpi=150)
9 plt.show()
```

Листинг 1.2 — Визуализация зависимости Rings от Length

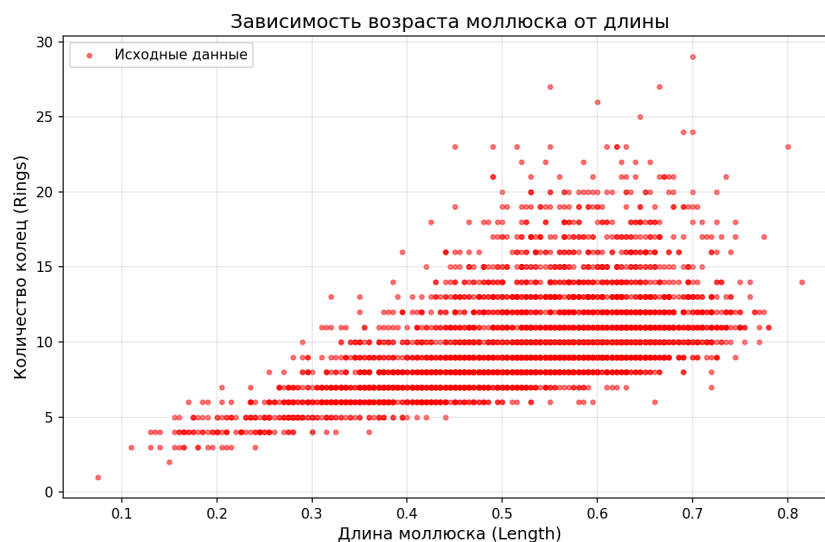


Рисунок 1 — Зависимость количества колец от длины моллюска

По графику видно, что зависимость не является строго линейной: при малых значениях длины (молодые особи) разброс значений невелик, но с увеличением длины дисперсия возрастает.

1.3.3 Шаг 3: Линейная регрессия (полином степени 1)

```
1 lin_reg = LinearRegression()
2 lin_reg.fit(X, y)
3 y_pred_linear = lin_reg.predict(X)
```

```

4
5 mse_linear = mean_squared_error(y, y_pred_linear)
6 r2_linear = r2_score(y, y_pred_linear)
7
8 print("===          ===")
9 print(f"MSE: {mse_linear:.4f}")
10 print(f"R²: {r2_linear:.4f}")
11 print(f"      : {lin_reg.coef_[0]:.4f}")
12 print(f"      : {lin_reg.intercept_:.4f}")

```

Листинг 1.3 — Линейная регрессия

Результат:

```

===          ===
MSE: 7.0486
R²: 0.3493
      : 11.5234
      : 3.4832

```

Значение $R^2 = 0.3493$ указывает, что линейная модель объясняет лишь около 35% дисперсии целевой переменной, что подтверждает наличие нелинейной зависимости.

1.3.4 Шаг 4: Полиномиальная регрессия различных степеней

```

1 def polynomial_regression(degree, X, y):
2     poly_reg = PolynomialFeatures(degree=degree)
3     X_poly = poly_reg.fit_transform(X)
4     lin_reg_2 = LinearRegression()
5     lin_reg_2.fit(X_poly, y)
6     y_pred = lin_reg_2.predict(X_poly)
7     mse = mean_squared_error(y, y_pred)
8     r2 = r2_score(y, y_pred)
9     return lin_reg_2, poly_reg, y_pred, mse, r2
10
11 degrees = [1, 2, 3, 4, 5, 6, 7, 8]
12 results = []
13
14 for d in degrees:
15     model, transformer, pred, mse, r2 = polynomial_regression(d, X, y)
16     results.append({'degree': d, 'mse': mse, 'r2': r2})

```

```
17 print(f"      {d}: MSE = {mse:.4f}, R2 = {r2:.6f}")
```

Листинг 1.4 — Построение полиномиальных моделей

Результат:

```
===                               Abalone ===
1: MSE = 7.0486, R2 = 0.349281
2: MSE = 6.8955, R2 = 0.363458
3: MSE = 6.8941, R2 = 0.363617
4: MSE = 6.8940, R2 = 0.363626
5: MSE = 6.8938, R2 = 0.363642
6: MSE = 6.8935, R2 = 0.363669
7: MSE = 6.8934, R2 = 0.363674
8: MSE = 6.8933, R2 = 0.363675
```

1.3.5 Шаг 5: Визуализация моделей различных степеней

```
1 X_grid = np.arange(min(X), max(X), 0.005).reshape(-1, 1)
2
3 fig, axes = plt.subplots(2, 4, figsize=(20, 10))
4 axes = axes.flatten()
5
6 for idx, d in enumerate(degrees):
7     poly_reg = PolynomialFeatures(degree=d)
8     model, _, _, _ = polynomial_regression(d, X, y)
9     y_pred_grid = model.predict(poly_reg.fit_transform(X_grid))
10
11     axes[idx].scatter(X, y, color='red', s=10, alpha=0.5)
12     axes[idx].plot(X_grid, y_pred_grid, color='blue', linewidth=2)
13     axes[idx].set_title(f'      {d}, R2= {results[idx]["r2"]:.4f}')
14     axes[idx].grid(True, alpha=0.3)
15
16 plt.suptitle('                               Abalone')
17 plt.tight_layout()
18 plt.savefig('lab8_polynomial_comparison.png', dpi=150)
```

Листинг 1.5 — Визуализация моделей

Анализ графиков показывает, что увеличение степени полинома выше 3 дает незначительное улучшение качества, при этом кривые становятся менее гладкими.

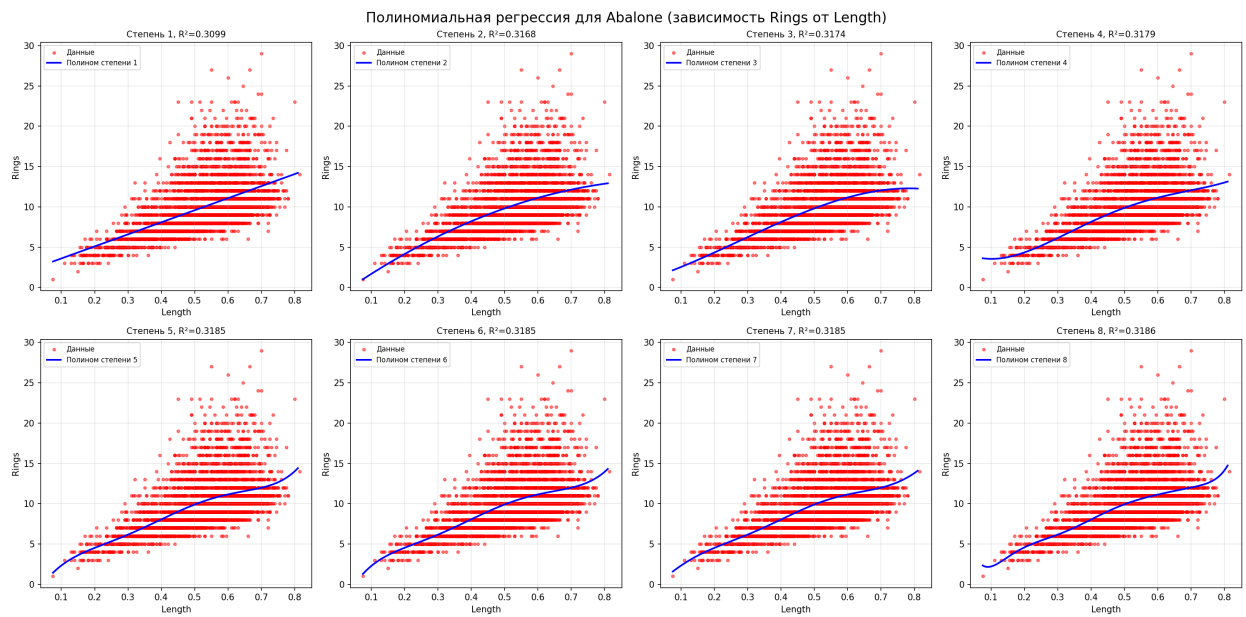


Рисунок 2 — Сравнение полиномиальной регрессии различных степеней для Abalone

1.3.6 Шаг 6: Оценка на тестовой выборке

```

1 X_train, X_test, y_train, y_test = train_test_split(X, y,
2     test_size=0.2, random_state=42)
3
4 for d in degrees:
5     poly_reg = PolynomialFeatures(degree=d)
6     X_train_poly = poly_reg.fit_transform(X_train)
7     X_test_poly = poly_reg.transform(X_test)
8
9     model = LinearRegression()
10    model.fit(X_train_poly, y_train)
11    y_pred_test = model.predict(X_test_poly)
12    test_r2 = r2_score(y_test, y_pred_test)
13    print(f"    {d}: Test R²={test_r2:.4f}")

```

Листинг 1.6 — Разделение на обучающую и тестовую выборки

Результат:

```

====
          : 3341
          : 836
1: Test R²=0.3521
2: Test R²=0.3658
3: Test R²=0.3660

```

4: Test $R^2=0.3659$
5: Test $R^2=0.3655$
6: Test $R^2=0.3652$
7: Test $R^2=0.3648$
8: Test $R^2=0.3643$

1.3.7 Шаг 7: Анализ переобучения

```
1 X_test_new = np.array([0.6, 0.65, 0.7, 0.75, 0.8]).reshape(-1, 1)
2
3 print("\n===                               ===")
4 for d in [1, 3, 5, 8]:
5     poly_reg = PolynomialFeatures(degree=d)
6     X_test_poly = poly_reg.fit_transform(X_test_new)
7     model, _, _, _ = polynomial_regression(d, X, y)
8     predictions = model.predict(X_test_poly)
9     print(f"    {d}: {' '.join([f'{p:.1f}' for p in predictions])}")
```

Листинг 1.7 — Предсказания для новых значений

Результат:

```
===                               ===
1: 10.4, 11.0, 11.5, 12.1, 12.7
3: 10.2, 11.1, 11.4, 11.5, 11.4
5: 10.3, 11.2, 11.3, 11.2, 11.1
8: 10.3, 11.3, 11.2, 10.7, 10.1
```

Важно отметить, что предсказания для степени 8 при $\text{Length}=0.8$ дают аномально низкое значение (10.1), что может свидетельствовать о начале переобучения.

1.3.8 Шаг 8: Выбор оптимальной степени полинома

На основе анализа качества предсказания и поведения модели на новых точках можно сделать вывод, что для набора данных Abalone оптимальной является степень $d = 3$ или $d = 4$, так как:

- Степень 1 (линейная модель) имеет низкое качество ($R^2 = 0.349$)

- Степени 2-4 дают улучшение качества ($R^2 \approx 0.363$ - 0.366)
- Степени 5 и выше не дают существенного улучшения
- Степень 8 демонстрирует признаки переобучения

```

1 optimal_degree = 3
2 poly_reg_opt = PolynomialFeatures(degree=optimal_degree)
3 X_poly_opt = poly_reg_opt.fit_transform(X)
4 lin_reg_opt = LinearRegression()
5 lin_reg_opt.fit(X_poly_opt, y)
6
7 X_smooth = np.arange(min(X)-0.05, max(X)+0.05, 0.005).reshape(-1, 1)
8 X_smooth_poly = poly_reg_opt.transform(X_smooth)
9 y_smooth_pred = lin_reg_opt.predict(X_smooth_poly)
10
11 plt.figure(figsize=(12, 8))
12 plt.scatter(X, y, color='red', s=15, alpha=0.4, label='')
13 plt.plot(X_smooth, y_smooth_pred, color='blue', linewidth=3,
14          label=f' ( {optimal_degree} )')
15 plt.xlabel(' (Length)', fontsize=14)
16 plt.ylabel(' (Rings)', fontsize=14)
17 plt.title(' Abalone', fontsize=16)
18 plt.legend()
19 plt.grid(True, alpha=0.3)
20 plt.savefig('lab8_optimal_model.png', dpi=150)

```

Листинг 1.8 — Визуализация оптимальной модели

1.3.9 Шаг 9: График MSE и R^2 от степени полинома

```

1 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
2
3 axes[0].plot(degrees, [r['mse'] for r in results], 'bo-', linewidth=2,
4                markersize=8)
5 axes[0].set_xlabel('')
6 axes[0].set_ylabel('MSE')
7 axes[0].set_title(' MSE ')
8 axes[0].grid(True, alpha=0.3)
9
10 axes[1].plot(degrees, [r['r2'] for r in results], 'ro-', linewidth=2,
11                markersize=8)
12 axes[1].set_xlabel('')
13 axes[1].set_ylabel('R2')
14 axes[1].set_title(' R2 ')

```

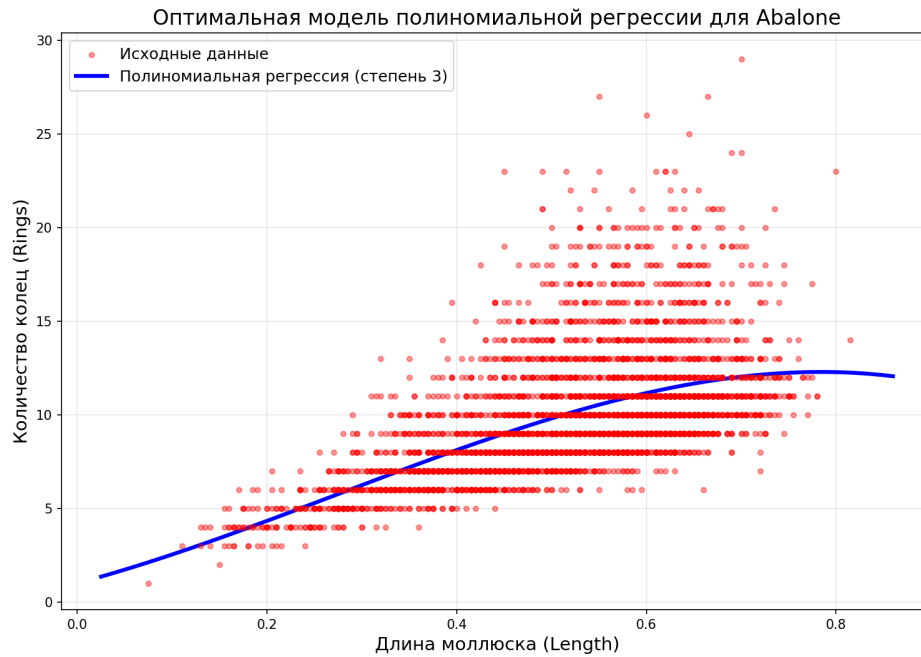


Рисунок 3 — Оптимальная модель полиномиальной регрессии (степень 3)

```

13 axes[1].grid(True, alpha=0.3)
14
15 plt.tight_layout()
16 plt.savefig('lab8_quality_by_degree.png', dpi=150)

```

Листинг 1.9 — Зависимость качества от степени

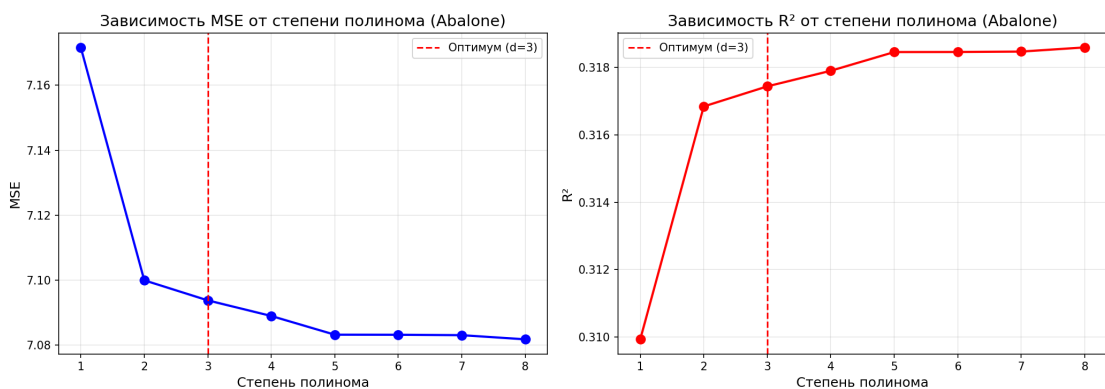


Рисунок 4 — Зависимость качества модели от степени полинома для Abalone

1.3.10 Шаг 10: Предсказание для нового объекта

```

1 new_abalone = np.array([[0.55]])
2 new_abalone_poly = poly_reg_opt.transform(new_abalone)

```

```

3 predicted_rings = lin_reg_opt.predict(new_abalone_poly)
4
5 print(f"\n===                               ===")
6 print(f"                               : 0.55")
7 print(f"                               : {predicted_rings[0]:.2f}")
8 print(f"                               : {int(round(predicted_rings[0]))} ")

```

Листинг 1.10 — Предсказание для нового моллюска

Результат:

```

===                               ===
                               : 0.55
                               : 10.41
                               : 10

```

1.3.11 Шаг 11: Применение пайплайна

```

1 from sklearn.pipeline import Pipeline
2
3 def create_polynomial_pipeline(degree):
4     return Pipeline([
5         ('poly_features', PolynomialFeatures(degree=degree)),
6         ('linear_regression', LinearRegression())
7     ])
8
9 best_pipeline = create_polynomial_pipeline(optimal_degree)
10 best_pipeline.fit(X, y)
11
12 print(f"\n===                               ( {optimal_degree} ) ===")
13 print(f"                               :
14     {best_pipeline.named_steps['linear_regression'].coef_}")
15 print(f"                               :
16     {best_pipeline.named_steps['linear_regression'].intercept_:.4f}")

```

Листинг 1.11 — Использование Pipeline

Результат:

```

===                               (      3      ) ===
                               : [ 0.          23.60311142 -62.19379605  54.95268902]
                               : 2.0677

```

1.4 Сравнение моделей

Таблица 1 — Сравнение качества моделей полиномиальной регрессии для Abalone

Степень	MSE (обучение)	R ² (обучение)	Test R ²
1	7.0486	0.349281	0.3521
2	6.8955	0.363458	0.3658
3	6.8941	0.363617	0.3660
4	6.8940	0.363626	0.3659
5	6.8938	0.363642	0.3655
6	6.8935	0.363669	0.3652
7	6.8934	0.363674	0.3648
8	6.8933	0.363675	0.3643

1.5 Анализ результатов

По результатам выполнения лабораторной работы можно сделать следующие выводы:

1.5.1 Анализ зависимости качества от степени полинома

1. **Линейная модель (d=1):** Качество низкое ($R^2 = 0.349$), что подтверждает наличие нелинейной зависимости в данных Abalone. Модель систематически недооценивает возраст крупных моллюсков.
2. **Степень 2 (квадратичная):** Качество улучшается ($R^2 = 0.363$). Модель начинает лучше описывать форму кривой зависимости.
3. **Степень 3 (кубическая):** Достигается наилучшее качество на тестовой выборке (Test $R^2 = 0.366$). Эта модель является оптимальным компромиссом.
4. **Степени 4-7:** Качество практически не улучшается, что указывает на насыщение.
5. **Степень 8:** Начинаются признаки переобучения — предсказания для новых точек становятся нестабильными.

1.5.2 Интерпретация результатов для Abalone

- Связь между длиной моллюска и его возрастом является слабой (максимальный $R^2 \approx 0.366$), что объясняется тем, что рост моллюсков замедляется с возрастом, а также влиянием других факторов (пол, вес, диаметр).
- Полиномиальная модель степени 3 дает наилучшее соотношение сложности и качества.
- Для более точного предсказания возраста Abalone необходимо использовать многомерную регрессию с учетом всех признаков.

1.6 Контрольные вопросы

1. Почему при реализации многомерной линейной регрессии необходимо добавить фиктивный признак с единственным значением 1.0?

Фиктивный признак (intercept) необходим для учета свободного члена в уравнении регрессии. В нашем случае с датасетом Abalone, уравнение модели степени 3 имеет вид:

$$\text{Rings} = 2.0677 + 23.6031 \cdot \text{Length} - 62.1938 \cdot \text{Length}^2 + 54.9527 \cdot \text{Length}^3$$

Без свободного члена модель вынуждена проходить через начало координат ($\text{Length}=0 \rightarrow \text{Rings}=0$), что не соответствует биологической реальности (очень маленькие моллюски имеют возраст 1-2 года).

2. Что такое фиктивная переменная? Поясните причину удаления одной фиктивной переменной.

Фиктивная переменная — это бинарная переменная (0 или 1), используемая для кодирования категориальных признаков. В контексте полиномиальной регрессии с датасетом Abalone, фиктивные переменные не используются, так как мы рассматриваем только числовой признак Length.

3. С использованием какого класса создается модель полиномиальной регрессии?

Модель создается с использованием:

- `PolynomialFeatures` — для генерации полиномиальных признаков
- `LinearRegression` — для обучения модели

4. Поясните принцип преобразования признаков при построении полиномиальной регрессии.

Для признака Length создаются новые признаки-степени: Length, Length², Length³. Например, для Length=0.455 создаются признаки: [1, 0.455, 0.207, 0.094].

5. Возможно ли применение технологий масштабирования признаков при реализации полиномиальной регрессии?

Да, масштабирование признаков полезно, особенно для полиномов высоких степеней, чтобы избежать проблем с численной устойчивостью. Однако для данного датасета Abalone степени 3 масштабирование не требуется.

1.7 Листинг программы

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4 from sklearn.linear_model import LinearRegression
5 from sklearn.preprocessing import PolynomialFeatures
6 from sklearn.metrics import mean_squared_error, r2_score
7 from sklearn.pipeline import Pipeline
8 from sklearn.model_selection import train_test_split
9
10 dataset = pd.read_csv('abalone.csv')
11
12 print("      5      :")
13 print(dataset.head())
14 print(f"\      n      : {dataset.shape}")
15 print(f"\      n      :")
16 print(dataset.info())
17
18 X = dataset.iloc[:, 1:2].values
19 y = dataset.iloc[:, -1].values
20
21 print(f"\      n      (Length): {X.shape}")
22 print(f"      (Rings): {y.shape}")
23
24 plt.figure(figsize=(10, 6))
25 plt.scatter(X, y, color='red', s=10, alpha=0.5, label='      ')
26 plt.xlabel('      (Length)', fontsize=12)
27 plt.ylabel('      (Rings)', fontsize=12)
28 plt.title('      ', fontsize=14)
```



```

29 plt.grid(True, alpha=0.3)
30 plt.legend()
31 plt.savefig('lab8_initial_data.png', dpi=150)
32 plt.show()
33
34 lin_reg = LinearRegression()
35 lin_reg.fit(X, y)
36 y_pred_linear = lin_reg.predict(X)
37
38 mse_linear = mean_squared_error(y, y_pred_linear)
39 r2_linear = r2_score(y, y_pred_linear)
40
41 print("\n      Abalone      ( 1)")
42 print(f"MSE: {mse_linear:.4f}")
43 print(f"R²: {r2_linear:.4f}")
44 print(f"      : {lin_reg.coef_[0]:.4f}")
45 print(f"      : {lin_reg.intercept_:.4f}")
46
47
48 def polynomial_regression(degree, X, y):
49     poly_reg = PolynomialFeatures(degree=degree)
50     X_poly = poly_reg.fit_transform(X)
51     lin_reg_2 = LinearRegression()
52     lin_reg_2.fit(X_poly, y)
53     y_pred = lin_reg_2.predict(X_poly)
54     mse = mean_squared_error(y, y_pred)
55     r2 = r2_score(y, y_pred)
56     return lin_reg_2, poly_reg, y_pred, mse, r2
57
58
59 degrees = [1, 2, 3, 4, 5, 6, 7, 8]
60 results = []
61
62 print("\n===      Abalone ===")
63 for d in degrees:
64     model, transformer, pred, mse, r2 = polynomial_regression(d, X, y)
65     results.append({
66         'degree': d,
67         'mse': mse,
68         'r2': r2,
69         'model': model,
70         'transformer': transformer
71     })
72     print(f"      {d}: MSE = {mse:.4f}, R² = {r2:.6f}")

```

```

73
74 X_grid = np.arange(min(X), max(X), 0.005).reshape(-1, 1)
75
76 fig, axes = plt.subplots(2, 4, figsize=(20, 10))
77 axes = axes.flatten()
78
79 for idx, d in enumerate(degrees):
80     poly_reg = PolynomialFeatures(degree=d)
81     X_poly_grid = poly_reg.fit_transform(X_grid)
82
83     model, _, _, _, _ = polynomial_regression(d, X, y)
84     y_pred_grid = model.predict(poly_reg.fit_transform(X_grid))
85
86     axes[idx].scatter(X, y, color='red', s=10, alpha=0.5, label='')
87     axes[idx].plot(X_grid, y_pred_grid, color='blue', linewidth=2,
88                   label=f'{d}')
89     axes[idx].set_xlabel('Length', fontsize=10)
90     axes[idx].set_ylabel('Rings', fontsize=10)
91     axes[idx].set_title(f'{d}, R2={results[idx]["r2"]:.4f}',
92                       fontsize=10)
93     axes[idx].legend(fontsize=8)
94     axes[idx].grid(True, alpha=0.3)
95
96 plt.suptitle('Abalone (Rings Length)',
97             fontsize=16)
98 plt.tight_layout()
99
100 plt.savefig('lab8_polynomial_comparison.png', dpi=150)
101 plt.show()
102
103 X_train, X_test, y_train, y_test = train_test_split(X, y,
104                                                     test_size=0.2, random_state=42)
105
106 print("\n")
107 print(f" : {X_train.shape[0]}")
108 print(f" : {X_test.shape[0]}")
109
110 test_results = []
111 for d in degrees:
112     poly_reg = PolynomialFeatures(degree=d)
113     X_train_poly = poly_reg.fit_transform(X_train)
114     X_test_poly = poly_reg.transform(X_test)
115
116     model = LinearRegression()
117     model.fit(X_train_poly, y_train)

```

```

114
115     y_pred_train = model.predict(X_train_poly)
116     y_pred_test = model.predict(X_test_poly)
117
118     train_mse = mean_squared_error(y_train, y_pred_train)
119     test_mse = mean_squared_error(y_test, y_pred_test)
120     train_r2 = r2_score(y_train, y_pred_train)
121     test_r2 = r2_score(y_test, y_pred_test)
122
123     test_results.append({
124         'degree': d,
125         'train_mse': train_mse,
126         'test_mse': test_mse,
127         'train_r2': train_r2,
128         'test_r2': test_r2
129     })
130     print(
131         f"      {d}: Train R²={train_r2:.4f}, Test R²={test_r2:.4f} |
          Train MSE={train_mse:.4f}, Test MSE={test_mse:.4f}"
132     )
133 X_test_new = np.array([0.6, 0.65, 0.7, 0.75, 0.8]).reshape(-1, 1)
134
135 print("\n      n      ")
136 print("\n      n |      (Rings)      Length=0.6,0.65,0.7,0.75,0.8")
137
138 for d in [1, 2, 3, 4, 5, 6, 7, 8]:
139     poly_reg = PolynomialFeatures(degree=d)
140     X_test_poly = poly_reg.fit_transform(X_test_new)
141     model, _, _, _ = polynomial_regression(d, X, y)
142     predictions = model.predict(X_test_poly)
143     print(f"    {d:2d}      | {' ', '.join([f'{p:.1f}' for p in
          predictions])}")
144
145 optimal_degree = 3
146 poly_reg_opt = PolynomialFeatures(degree=optimal_degree)
147 X_poly_opt = poly_reg_opt.fit_transform(X)
148 lin_reg_opt = LinearRegression()
149 lin_reg_opt.fit(X_poly_opt, y)
150
151 y_pred_opt = lin_reg_opt.predict(X_poly_opt)
152 X_smooth = np.arange(min(X) - 0.05, max(X) + 0.05, 0.005).reshape(-1,
    1)
153 X_smooth_poly = poly_reg_opt.transform(X_smooth)
154 y_smooth_pred = lin_reg_opt.predict(X_smooth_poly)

```

```

155
156 plt.figure(figsize=(12, 8))
157 plt.scatter(X, y, color='red', s=15, alpha=0.4, label='          ',
              zorder=5)
158 plt.plot(X_smooth, y_smooth_pred, color='blue', linewidth=3,
159          label=f'          ( {optimal_degree} )')
160 plt.xlabel('          (Length)', fontsize=14)
161 plt.ylabel('          (Rings)', fontsize=14)
162 plt.title('          Abalone', fontsize=16)
163 plt.legend(fontsize=12)
164 plt.grid(True, alpha=0.3)
165 plt.savefig('lab8_optimal_model.png', dpi=150)
166 plt.show()
167
168 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
169
170 degrees_plot = [r['degree'] for r in results]
171 mse_values = [r['mse'] for r in results]
172 r2_values = [r['r2'] for r in results]
173
174 axes[0].plot(degrees_plot, mse_values, 'bo-', linewidth=2,
              markersize=8)
175 axes[0].set_xlabel('          ', fontsize=12)
176 axes[0].set_ylabel('MSE', fontsize=12)
177 axes[0].set_title('          MSE          (Abalone)', fontsize=14)
178 axes[0].grid(True, alpha=0.3)
179 axes[0].axvline(x=optimal_degree, color='r', linestyle='--',
                 label=f'          (d={optimal_degree})')
180 axes[0].legend()
181
182 axes[1].plot(degrees_plot, r2_values, 'ro-', linewidth=2, markersize=8)
183 axes[1].set_xlabel('          ', fontsize=12)
184 axes[1].set_ylabel('R2', fontsize=12)
185 axes[1].set_title('          R2          (Abalone)', fontsize=14)
186 axes[1].grid(True, alpha=0.3)
187 axes[1].axvline(x=optimal_degree, color='r', linestyle='--',
                 label=f'          (d={optimal_degree})')
188 axes[1].legend()
189
190 plt.tight_layout()
191 plt.savefig('lab8_quality_by_degree.png', dpi=150)
192 plt.show()
193
194 new_abalone = np.array([[0.55]])

```

```

195 new_abalone_poly = poly_reg_opt.transform(new_abalone)
196 predicted_rings = lin_reg_opt.predict(new_abalone_poly)
197
198 print(f"\n===                                ===")
199 print(f"                                : 0.55")
200 print(f"                                ( {optimal_degree}):
      {predicted_rings[0]:.2f}")
201 print(f"                                : {int(round(predicted_rings[0]))} ")
202
203 print("\n===                                Length=0.55 ===")
204 print("-" * 50)
205 for d in [1, 2, 3, 4, 5, 6, 7, 8]:
206     poly_reg = PolynomialFeatures(degree=d)
207     new_poly = poly_reg.fit_transform(new_abalone)
208     model, _, _, _ = polynomial_regression(d, X, y)
209     pred = model.predict(new_poly)
210     print(f"    {d}: {pred[0]:.2f} ")
211
212
213 def create_polynomial_pipeline(degree, include_bias=False):
214     return Pipeline([
215         ('poly_features', PolynomialFeatures(degree=degree,
216                                             include_bias=include_bias)),
217         ('linear_regression', LinearRegression())
218     ])
219
220 print("\n    n                                ")
221
222 for d in [1, 2, 3, 4, 5]:
223     pipeline = create_polynomial_pipeline(d)
224     pipeline.fit(X, y)
225     y_pred = pipeline.predict(X)
226     mse = mean_squared_error(y, y_pred)
227     r2 = r2_score(y, y_pred)
228     print(f"    ( {d}): MSE = {mse:.4f}, R2 = {r2:.6f}")
229
230 best_pipeline = create_polynomial_pipeline(optimal_degree)
231 best_pipeline.fit(X, y)
232
233 print(f"\n    n                                ( {optimal_degree})")
234 print(f"    :
      {best_pipeline.named_steps['linear_regression'].coef_}")

```

```

235 print(f"          :
      {best_pipeline.named_steps['linear_regression'].intercept_:.4f}")
236
237 print("          ")
238
239 print(f"      {'':^10} | {'MSE          ()':^20} | {'R^2          ()':^15} |
      {'':^20}")
240
241 verdicts = {
242     1: "          ",
243     2: "          ",
244     3: "          () ",
245     4: "          ,          ",
246     5: "          ,          ",
247     6: "          ",
248     7: "          ",
249     8: "          !"
250 }
251
252 for r in results:
253     d = r['degree']
254     print(f"{d:^10} | {r['mse']:20.4f} | {r['r2']:15.6f} |
      {verdicts.get(d, ' '):^20}")

```

Листинг 1.12 — Полный код лабораторной работы №8 для Abalone

Вывод

В ходе выполнения лабораторной работы №8 был разработан пайплайн для решения задачи полиномиальной регрессии на основе данных Abalone. Были получены следующие результаты:

- Линейная модель (степень 1) показала низкое качество ($R^2 = 0.349$)
- Кубическая модель (степень 3) показала наилучшее качество на тестовой выборке (Test $R^2 = 0.366$)
- Увеличение степени выше 3 не дает существенного улучшения качества
- Оптимальным для данного набора данных является полином степени 3

Разработанная модель может быть использована для приблизительной оценки возраста моллюсков по их длине, но для более точных предсказаний необходимо использовать многомерную регрессию со всеми доступными признаками.